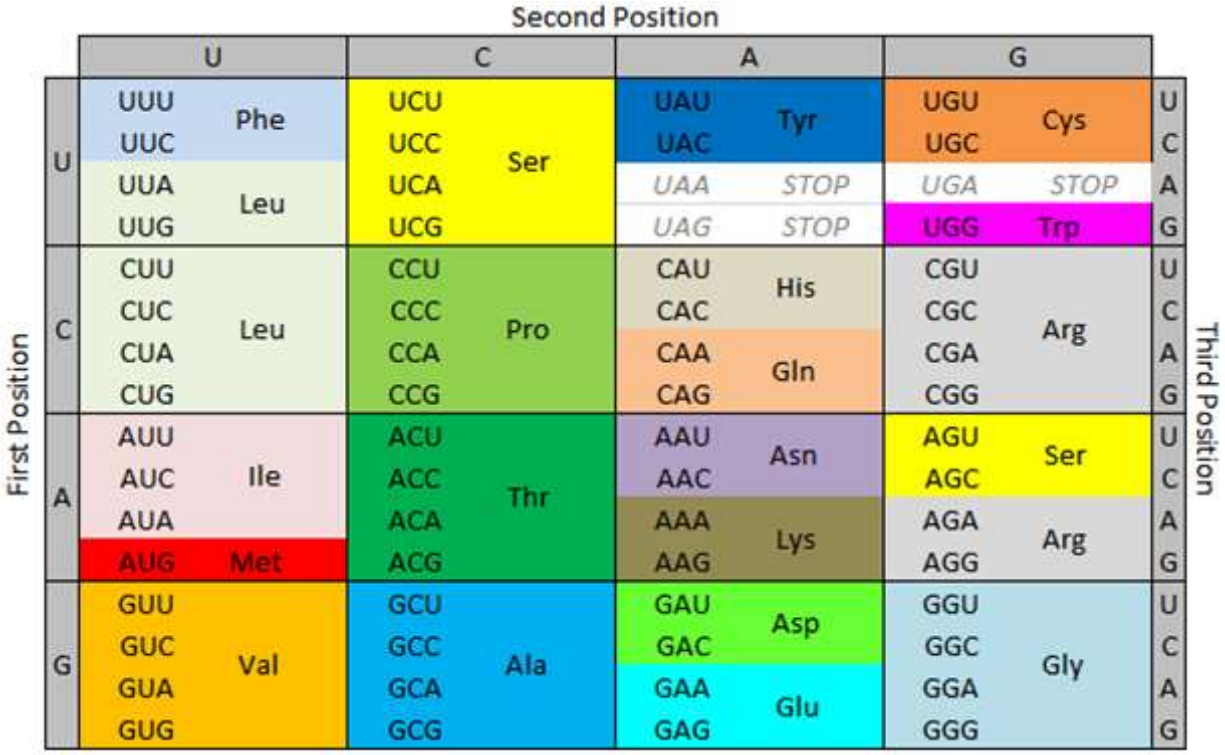
A

To Marry or Not to Marry?

Landia country is strict about its marriage policy, allowing only couples with genes that are quite different to marry. To avoid diseases, the similarity of their genes should not exceed 80%. Doctors take samples and process them, i.e., convert the DNA sequences to amino acids. The sample can be corrupted; it must be checked if it was collected correctly: the DNA sequence can be converted to amino acids. A DNA sequence is correct if it only contains the 4 nucleotides (letters): A, G, T, and C. The sequence must begin with the Methionine (Met in the table) amino acid. Nucleotides can be converted to amino acids without residue and there must be only one stop signal in the sequence at the end of it. The DNA is transformed as follows: 3 nucleotides are converted to one amino acid according to the following table:



Determine how similar the chains of the two amino acids are. If they are similar in more than 80%, the program should output "`Forbidden`", otherwise "`Allowed`". In case of incorrect input, the program should warn us with the following message: "`Incorrect input`".

The similarity of two chains of amino acids means how many amino acids need to be replaced, deleted, or added to convert one into the other. That is, the similarity is the ratio of the number of steps to the length of the longer sequence, excluding the first and last amino acids:

$$1 - \frac{steps}{\max\{amino_1, amino_2\} - 2}$$

Input Specification

The input contains two lines in varying lengths (between 0 and 120), the DNA sequences. The DNA sequences aren't necessarily the same length.

Output Specification

Print one line to the standard output, containing one of "`Forbidden`", "`Allowed`", and "`Incorrect input`", as specified above.

Sample Input 1

```
AUGUCUGAUAGUUAA
AUGAUAGUAUAG
```

Output for Sample Input 1

```
Allowed
```

Sample Input 2

```
AUGUCUUAGUAA
AUGAUAUCUUAG
```

Output for Sample Input 2

```
Incorrect input
```

Sample Input 3

```
AUGUCUUUUAUUAAUGGAAUACAUUAA
AUGUCUUUUAUUAAUGGAGUGCAUUAG
```

Output for Sample Input 3

```
Forbidden
```

B

Ecstasy of Gold

Dirty Fred and his ferocious gang successfully broke into the treasury of Goliath Multicontinental Bank at last! Inside the treasury they found a deposit of gold bars. The bars have similar (though not necessarily equal) weights, but at the same time their respective values can differ a lot, based on how fine the gold is. Formally, every gold bar has two characteristics: its weight w_i and its value v_i, where the difference between the weights of the lightest and the heaviest bars does not exceed 3.

Luckily for them, Fred's trained eyes can measure the value of any gold bar immediately. There is a problem though: the members of the gang — even though they trained hard — have a combined strength of W, meaning that they can carry at most W units of weight in total.

This capacity may not be enough to take every gold bar with them, so your task is to help them determine the maximum total value of gold bars they can steal, provided the total weight of the stolen bars does not exceed W.

Input Specification

The first line of the input contains the number of gold bars N ($1 \le N \le 100$) and the combined strength of the gang W ($1 \le W \le 10^9$).

Each of the next N lines contains two integers w_i and v_i, describing one of the gold bars ($1 \le w_i \le 10^9$, $1 \le v_i \le 10^7$). The difference between the highest and lowest weights w_{max} and w_{min} is at most 3, i.e., $w_{max} \le w_{min} + 3$.

Output Specification

Print a single line containing the highest total value of gold bars they can steal, provided the total weight of the stolen bars does not exceed W.

Sample Input 1

```
4 6
2 1
3 4
4 10
3 4
```

Output for Sample Input 1

```
11
```

Sample Input 2

```
4 6
2 1
3 7
4 10
3 6
```

Output for Sample Input 2

```
13
```

Sample Input 3

```
1 10
20 100
15 200
```

Output for Sample Input 3

```
0
```

In the first example, they can steal the first and the third bar, as their total weight is 2 + 4 = 6. In this case, the total value of the stolen bars is 1 + 10 = 11.



Kings on a Chessboard

Place a maximum number of kings on an $n \times n$ chessboard ($0 < n < 11$) so that no kings attack each other. The king attacks its adjoining squares. Determine the maximum number of kings which can be put on the chessboard and the number of possible ways they can be placed based on the input data.

Input Specification

The first line of the input contains an integer t, which is the number of test cases. The next t lines contain t different test cases with an integer n, representing n from the problem description.

Output Specification

The first line of a test case should contain the maximum number of kings which can be placed on the chessboard, the second line should contain the number of possibilities. Print an empty line between successive test cases.

Sample Input

```
3
2
4
3
```

Output for Sample Input

```
1
4

4
79

4
1
```

D

Mountain Problem

A mountain range has the property that among its N elements there is an index M inside the range $1 < M < N$, for which the sequence formed by the first M elements (a_1, a_2, …, a_M) is ascending and nonconstant, and the sequence consisting of the last $N – M + 1$ elements (a_M, a_{M+1}, …, a_N) is descending and nonconstant.

For example, the following 8 elements form a mountain range: {1, 1, 1, 2, 2, 2, 2, 1}.

These sequences with N elements are indexed from 1 to N, the elements being a_1, a_2, …, a_N. These are signed 32-bit integers, not necessarily distinct.

We can apply two types of symmetric swap operations on these elements:

1. **swap(*P*, *N* – *P* + 1)**: swap the elements on the positions P and $N – P + 1$, placed equidistantly from the two ends of the sequence ($P \neq N – P + 1$, $1 \leq P \leq N$);
2. **double_swap(*P*, *Q*, *N* – *P* + 1, *N* – *Q* + 1)**: swap simultaneously two pairs of elements. The elements from positions P and Q will be swapped simultaneously with the elements from positions $N – P + 1$ and $N – Q + 1$. The operation can be applied only if all four positions are distinct ($P \neq Q$, $P \neq N – P + 1$, $Q \neq N – Q + 1$, $Q \neq N – P + 1$, $1 \leq P \leq N$, $1 \leq Q \leq N$).

How many distinct mountain ranges can be achieved using any number of operations defined above?

Example 1

For $N = 7$ and $a = \{3,6,2,10,1,9,8\}$, we can obtain 4 distinct mountain ranges:

The first mountain range can be obtained from the sequence {3,6,2,10,1,9,8} with the following operations:

1. double_swap(1,5,7,3): the highlighted elements of {**3**,6,2,10,**1**,9,8} are swapped, and the mountain range of {1,6,8,10,3,9,2} is obtained;
2. double_swap(2,5,6,3): the highlighted elements of {1,**6**,8,10,**3**,9,2} are swapped, and the mountain range of {1,3,9,10,6,8,2} is obtained;
3. swap(3,5): the highlighted elements of {1,3,**9**,10,**6**,8,2} are swapped, and the mountain range of {1,3,6,10,9,8,2} is obtained.

The second mountain range obtained from {3,6,2,10,1,9,8} is:

1. double_swap(1,3,7,5): the range of {2,6,3,10,8,9,1} is obtained;
2. double_swap(2,3,6,5): the mountain range of {2,3,6,10,9,8,1} is obtained.

The third mountain range obtained from {3,6,2,10,1,9,8} is:

1. double_swap(2,5,6,3): the range of {3,1,9,10,6,2,8} is obtained;
2. double_swap(2,1,6,7): the range of {1,3,9,10,6,8,2} is obtained;
3. swap(2,6) to get the mountain range of {1,8,9,10,6,3,2}.

The fourth mountain range obtained from {3,6,2,10,1,9,8} is:

1. double_swap(7,2,1,6): the range of {9,8,2,10,1,3,6} is obtained;
2. double_swap(5,7,3,1) to get the mountain range of {2,8,9,10,6,3,1}.

Example 2

For $N = 8$ and $a = \{2,1,2,1,1,1,2,2\}$, with the help of swap and double_swap operations, we can get 2 distinct mountain ranges: {1,1,1,2,2,2,2,1} and {1,2,2,2,2,1,1,1}, respectively.

Input Specification

The input contains several sets of tests. Each test contains on its first line a number N, representing the length of the sequence ($3 \le N \le 2005$), and on the next line N natural numbers a_i, the elements of the mountain range ($0 \le a_i \le 2\,000\,000\,000$), separated by a space.

Output Specification

Print as many lines as there are sets of tests in the input. Each line should contain the results corresponding to the data in the input in the order in which they are read. For each read string, print the number of distinct mountain ranges that can be obtained by applying the swap and double_swap operations on the elements of the given ranges. As this value can be very high, the result should be printed in modulo 111 181 111.

Note that each test case has at least one solution.

Sample Input

```
8
2 1 2 1 1 1 2 2
7
3 6 2 10 1 9 8
3
1 2 1
9
1 1 2 2 1 1 1 100 1
```

Output for Sample Input

```
2
4
1
10
```

Explanation

For the first test case, there are two solutions: {1,1,1,2,2,2,2,1} and {1,2,2,2,2,1,1,1}.

For the second test case, there are four solutions: {2,3,6,10,9,8,1}, {2,8,9,10,6,3,1}, {1,8,9,10,6,3,2}, and {1,3,6,10,9,8,2}.

For the third test case, there is only one solution: {1,2,1}.

For the fourth test case, there are ten solutions: {1,1,1,1,1,1,2,100,2}, {1,1,1,1,1,1,100,2,2}, {1,1,1,1,1,2,2,100,1}, {1,1,1,1,1,2,100,2,1}, {1,1,1,1,1,100,2,2,1}, {1,2,2,100,1,1,1,1,1}, {1,2,100,2,1,1,1,1,1}, {1,100,2,2,1,1,1,1,1}, {2,2,100,1,1,1,1,1,1}, and {2,100,2,1,1,1,1,1,1}.

E

Supervising the National Elections

The secret service of Landia has been empowered to supervise the national elections. The secret service uses its own database of electors, which is described as follows (standard input):

- on the first line, the number of electors n ($n \leq 150\,000$);
- on the next n lines, personal code (31-bit positive integer) and name (≤ 27 characters including white space) of electors.

Following the end of this list, the standard input includes an unspecified number (≤ 200 000) of personal codes of those who want to vote. **Beware:** some persons are not registered (personal code not found in the first list), and some registered persons try to vote twice.

We guarantee that: if a person is not registered, it appears only once in the second list; if a registered person wants to vote many times, it appears only twice in the second list.

Every line of the second list includes an arbitrary number of codes, separated by white spaces.

After the end of elections, the secret service publishes the following statistics (standard output):

- presence at the election process (number and percent related to the number of electors);
- number of notregistered persons;
- frauds (number and percent related to the presence).

The list of shame (fraudulent persons, ≤ 20) is also published, alphabetically ordered.

Sample Input

```
5
32 Tamp Ken
16 Kong Kim
20 Tram Tim
45 Endo Joe
11 Faro Mia
10 21 20 11
32 20 32 16
```

Output for Sample Input

```
Presence: 4 80.00%
Not registered: 2
Frauds: 2 50.00%
32 Tamp Ken
20 Tram Tim
```

Explanation

The codes 10 and 21 are not found in the first list (not registered). Four registered persons were present (20, 11, 32, 16). Two of them (20, 32) tried to vote twice (fraudulent).

F

Netflix

Netflix stocks are approaching an all-time low, so the company is forced to take desperate steps. The plan is to go back to the basics and hire traveling salesmen that will go door to door and will try to convince possible clients to subscribe to the service.

Your friend was out of job for some time now (in case making Tik-Tok videos doesn't count as one), so in order to pay the bills, he decided to try out this salesman thing at Netflix. He was assigned to a strange neighborhood, with one-way streets only, and having the property that starting from any house, there is no way to get back there. Your friend needs your help to determine if it's possible for him to knock on all the doors in the neighborhood.

Input Specification

The first line of the input contains t, the number of test cases to follow ($1 \le t \le 20$). Each test case starts with a line containing n, the number of houses in the neighborhood ($1 \le n \le 100\,000$), and m, the number of streets ($1 \le m \le 1\,000\,000$). The following m lines contain two numbers x and y, with the meaning that there is a street going from house x to house y. The houses are numbered from 1 to n. Whitespaces can occur freely in the input.

Output Specification

For each test case, you must output a single line. If your friend can visit all the houses, output a possible order in which he can visit them. Otherwise output -1.

Sample Input

```
4

3 2
1 3
3 2

4 4
4 2
2 3
4 1
1 3

4 2
1 2
3 4

5 6
2 1
1 3
3 5
1 4
2 4
4 3
```

Output for Sample Input

```
1 3 2
-1
-1
2 1 4 3 5
```

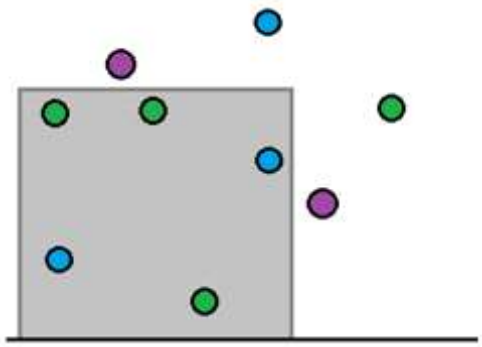
G

Plenty of Fish

The nature reserve of Lake Tisza is the home of K species of fish. There are N prominent habitats at different spots over the lake, where fishes can be found in great abundance — for simplicity, let us assume that each habitat houses a single species of fish.

The lake is also a favourite place of fishermen. To preserve the richness of nature, fishing in the lake is regulated: fishing is allowed only near the southern shore, inside a rectangular area specified by the Fishing Association. For this problem, let the x-axis of the Cartesian plane represent the shore of the lake, and the points above it represent the lake itself. Suppose that the fishing area is an axis-aligned rectangle and its bottom side rests on the x-axis.

Recently, there is a gossip that the Office for Environmental Protection wants to ban the fishing of one of the species — but no one knows which one yet! If this happens, the Association must be ready to select a new fishing area: one that has no habitats of the banned species at all. On the other hand, they want to choose an area containing as many habitats of any other (non-banned) species as possible.



An optimal fishing area (containing 5 habitats) when the species denoted by purple is banned.

The association wants you to help them prepare for each possible scenario. Your task is to determine for each species of fish that, if it was banned, then what is the maximal number of habitats an optimally selected fishing area could contain.

Input Specification

The first line of the input contains the number of test cases T.

Each test case starts with a line containing the number of habitats N ($2 \le N \le 200\,000$) and the number of species K ($2 \le K \le N$). The next N lines denote the positions of habitats (x_i, y_i) and the species s_i dwelling in the habitat ($1 \le x_i, y_i \le 10^9$, $1 \le s_i \le K$).

It is guaranteed that for each species, there is at least one habitat housing it, and that the total number of habitats over the T testcases does not exceed 10^6.

Output Specification

For each test case, print a single line containing K integers: the i-th being the maximal number of habitats an optimally chosen fishing area can contain in the case when fishing the i-th species is banned.

Sample Input

```
1
9 3
1 2 1
1 5 2
3 6 3
4 5 2
5 1 2
6 4 1
6 7 1
7 3 3
9 5 2
```

Output for Sample Input

```
3 3 5
```

The example corresponds to the figure in the task statement.

H

Advanced Speed Pizza

A local pizza restaurant offers pizza delivery services to customers directly to their locations. The restaurant receives orders from multiple couriers and delivers the pizza using scooters. Orders are taken in the order provided by each courier and assigned to a driver. Each driver will have a scooter and can only pick up a single pizza order for delivery to a location. After each delivery, the driver returns to the restaurant to pick up another order, if any available, until all orders are delivered. The destination of an order can be any location on the map but will never be the restaurant's location. Locations are linked by two-way roads with an associated cost, representing the time needed for a scooter to go from one location to another.

The restaurant manager wants to deliver the received orders using the available scooters while keeping the customers happy. The manager has the following information:

- a map of locations numbered from 1 to *l*, linked by two-way roads with a time cost;
- the location of the restaurant;
- the total number of available scooters;
- the number of couriers;
- for each courier, we know the number of orders and their location.

Of course, a customer is happy if the pizza arrives quickly. We define *starvation* as the time needed by a customer to receive the pizza. Therefore, the *overall starvation* represents the sum of starvation values for all customers from all couriers. The task of the manager is to find the value of *minimum overall starvation*.

When assigning orders to scooters, the manager has the following rules of thumb:

1. *The manager is greedy!* The manager will always assign the order with minimum cost.
2. *The manager is blind!* At each moment, he assigns only from orders that he sees, and he sees only the first order from each courier.
3. *The manager is weak!* At each moment, he can choose only one order and assign it to a scooter, although there could be two available scooters at the same time.
4. *The manager has a preference over couriers!* The order in which the couriers have been registered to the restaurant represents the preference order of the manager. Suppose two (or more) orders from distinct couriers have the same cost (i.e., delivery time). In this case, the manager will always prefer the order from the courier who registered earlier to the restaurant.

Input Specification

The first line of the input consists of numbers *l* and *r*, representing the number of locations (numbered from 1 to *l*) and the number of roads that link them. On the following *r* lines, the two-way roads, given in the form of location l_1, location l_2 and cost *c*, meaning that from location l_1 to location l_2 (and vice versa), the time to travel is *c*.

On the following line, the location of the restaurant *p* is given.

The next line has two numbers, representing the number of scooters *s* and number of couriers *c*.

Finally, we have two lines for each courier: one with the number of orders from that courier and the following line with locations received by the restaurant from that courier, separated by a space.

The order of the couriers represents the order of registration.

Input Limits and Constraints

- $1 < l < 100\,000$
- $1 < r < 150\,000$
- $1 < s \leq c < 100\,000$
- The locations are reachable from the restaurant.
- The *overall starvation* will fit in 64 bit.
- There is no limitation on the number of scooters on the road at any moment in time.
- The sum of overall orders from all couriers will be less than 100 000.

Output Specification

It should contain a single line representing the minimum value of *overall starvation*.

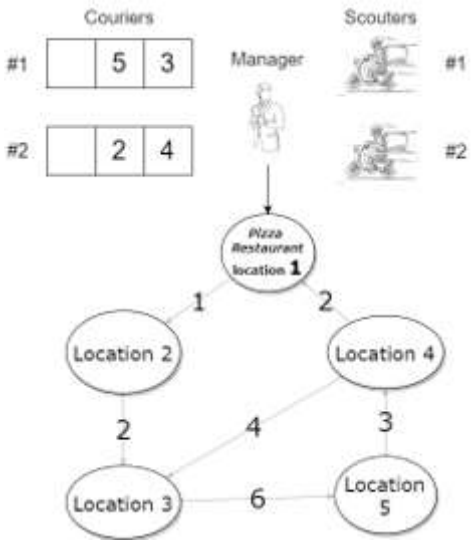
Sample Input

```
5 6
1 2 1
1 4 2
4 3 4
2 3 2
4 5 3
3 5 6
1
2 2
2
3 5
2
4 2
```

Output for Sample Input

```
17
```

Explanation



The time to deliver pizza to locations 2, 3, 4, and 5 is 1, 3, 2, and 5.

At Time 0 (both scooters are available)

Scooter #1 sees orders 3 (cost 3) and order 4 (cost 2).

*// The manager is **blind**, he sees only the fist orders from the couriers and chooses the minimum cost.*

Scooter #1 delivers order 4 with starvation value **2**.

Scooter #1 will be available at time 4.

*// The manager is **weak**, he can not assign both orders, he can assign only one. After assignment he sees order 2.*

Scooter #2 sees orders 3 (cost 3) and 2 (cost 1).

// These are now the top orders (order 4 is already assigned) of the couriers.

Scooter #2 delivers order 2 with starvation value **1**.

Scooter #2 will be available at time 2.

At Time 2 (only scooter #2 is available)

Scooter #2 sees order 3 (cost 3).

Scooter #2 delivers order 3 with starvation value **5**.

Scooter #2 will be available at time 8.

At Time 4 (only scooter #1 is available)

Scooter #1 sees order 5 (cost 5).

Scooter #1 delivers order 5 with starvation value **9**.

Scooter #1 will be available at time 22.

Overall starvation = 17 (2 + 1 + 5 + 9).

***Obs:** If the manager sees **more orders with the same cost**, he will prefer the first one he encounters when passing through the couriers in their order of registration. Couriers are registered in the same order they are given in the input.*



Tech Coding Challenge

Create a drawing program that uses the console as the canvas and characters as colors. You will take a list of commands and output the result in the console. The program should work as follows:

1. Create a new canvas.
2. Start drawing on the canvas by issuing various commands.
3. Quit.

The following commands should be interpreted:

- `C` *w h*: Should create a new canvas of width *w* and height *h*.
- `L` x_1 y_1 x_2 y_2: Should create a new line from (x_1,y_1) to (x_2,y_2). Currently only horizontal or vertical lines are supported. Horizontal and vertical lines will be drawn using the 'x' character.
- `R` x_1 y_1 x_2 y_2: Should create a new rectangle, whose upper left corner is (x_1,y_1) and lower right corner is (x_2,y_2). Horizontal and vertical lines will be drawn using the 'x' character.
- `F` *x y c*: Should fill the entire area connected to (*x*,*y*) with “colour” *c*. The behavior of this is the same as that of the “bucket fill” tool in paint programs.

The commands feed will be provided through the standard input, one command per line.

The result should be written to the standard output. Pay attention: the result must contain also the canvas borders.

Sample Input 1

```
C 20 4
```

Output for Sample Input 1

```
----------------------
|                    |
|                    |
|                    |
|                    |
----------------------
```

Sample Input 2

```
C 20 4
L 1 2 6 2
```

Output for Sample Input 2

```
----------------------
|                    |
|xxxxxx              |
|                    |
|                    |
----------------------
```

Sample Input 3

```
C 20 4
L 1 2 6 2
L 6 3 6 4
```

Output for Sample Input 3

```
----------------------
|                    |
|xxxxxx              |
|     x              |
|     x              |
----------------------
```

Sample Input 4

```
C 20 4
L 1 2 6 2
L 6 3 6 4
R 14 1 18 3
```

Output for Sample Input 4

```
----------------------
|             xxxxx  |
|xxxxxx       x   x  |
|     x       xxxxx  |
|     x              |
----------------------
```

Sample Input 5

```
C 20 4
L 1 2 6 2
L 6 3 6 4
R 14 1 18 3
F 10 3 o
```

Output for Sample Input 5

```
----------------------
|ooooooooooooxxxxxoo|
|xxxxxxoooooox   xoo|
|     xooooooxxxxxoo|
|     xooooooooooooo|
----------------------
```

J

Valuable Factorial

Let us define the *essential value* of a factorial $n! = 1 \cdot 2 \cdot \ldots \cdot n$ ($n \geq 1$) as the number of its trailing zeroes multiplied by the first nonzero digit before them.

For example, the essential value of $5! = 120$ is $1 \cdot 2 = 2$, while the essential value of $12! = 479\,001\,600$ is $2 \cdot 6 = 12$.

Given some number v, your task is to determine the smallest number n such that the essential value of $n!$ is exactly v, or report that such n does not exist.

Input Specification

The first line of the input contains the number of test cases T ($1 \leq T \leq 200\,000$).

Each test case consists of a single line specifying the essential value v ($1 \leq v \leq 10^{16}$).

Output Specification

For each test case, print a single line containing the smallest positive integer n such that the essential value of $n!$ is exactly v.

In the cases when such n does not exist, print –1.

Sample Input

```
7
1
2
4
12
24
1000000000
12345678910
```

Output for Sample Input

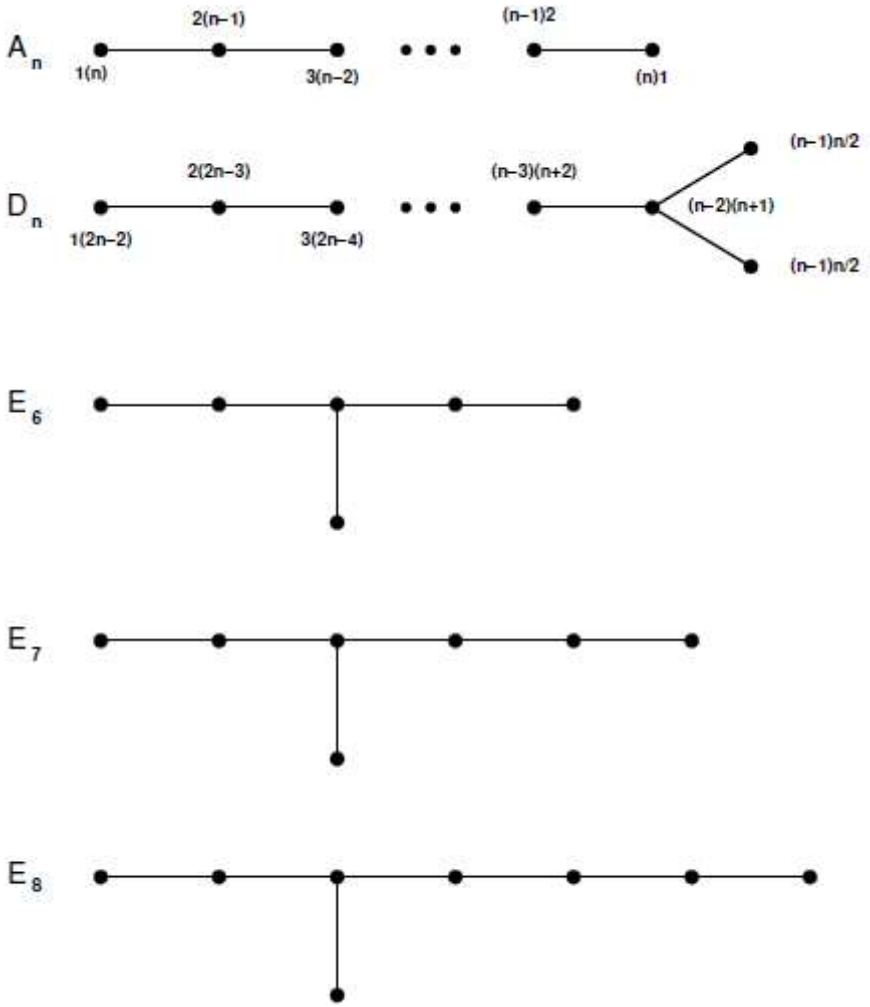
```
-1
5
7
12
15
500000009
24691357854
```

K

Graphs With a Special Labeling Property

In many distinct areas of mathematics, as a result of deep characterization theorems, surprisingly arrive — nobody knows exactly why — the same special graphs. A very simple instance in which these graphs also appear, is the next one.

Take a simple connected graph and label its vertices with positive integers in such a way that the *double of each number minus 2 is equal to the sum of the neighbouring numbers*. It turns out that such a labeling is never possible, except for a few very special graphs. These graphs are the members of two infinite family, having n vertices, named A_n ($n \geq 2$) and D_n ($n \geq 4$), shown below with the proper labeling, and surprisingly enough, three more singular graphs, having 6, 7, and 8 vertices respectively, named E_6, E_7, and E_8, shown below without their labeling.



Your task is to compute the proper labeling of these last three graphs.

Input Specification

There is no input for this problem.

Output Specification

Output three lines, each containing an integer, the sum of the labels of E_6, E_7, and E_8, respectively.

L

Antidote

In the 23rd century a virus with the codename ECN-2202 (**E**asy to **C**ure and **N**eutralize 2202) is making victims within the population. Mankind is heading towards social and economical disaster. A large research group is trying to find an antidote; they are using programming contests to find and run simulations of possibly good vaccines. The vaccine has two components: *VacA* and *VacB*. The research process results in n different *VacA* components and m different VacB components, described through their aggressiveness of the effect on the human organism. The components work in pairs, and every *VacA* is compatible with any of the *VacB* components, their aggressiveness is the cumulated aggressiveness of the two parts. The more aggressive the vaccine, the more effective it is. But too much aggressiveness can really hurt the body of the patient.

After many discussions, the scientists came up with a testing plan. They decide to leave the least effective vaccines apart, and to do so with the most aggressive ones too. To set the range, they set a lower and an upper bound for the effectiveness, but given that there can be up to 200 000 different components which can be combined, they can only set theoretical bounds. According to this, only those vaccines are eligible for testing which are strictly more effective than the Ath element and strictly less aggressive than the Bth element in the list of (*VacA*, *VacB*) in increasing ordered list of the cumulative aggressiveness. Given the increasingly ordered list of *VacA* and *VacB* components, help them figure out which combinations are in the tolerable range of aggressiveness, not to make many casualties in the trial.

Input Specification

The first line contains four space-separated integers: n is the cardinality of *VacA* components, m is the cardinality of *VacB* components, A is the index of the lower bound of the eligible combinations, B is the upper bound of the eligible combinations. The two indexes are considered in the increasing ordered list of all ($VacA_i$, $VacB_j$) combinations' cumulated aggressiveness ($1 \le i \le n$, $1 \le j \le m$).

The next n lines contain the *VacA* components' aggressiveness values as integers in increasing order.

The next m lines contain the *VacB* components' aggressiveness values as integers in increasing order.

Output Specification

The first line of the output should contain K, the total number of eligible vaccines.

The next K lines are formed by the index pairs of eligible ($VacA_i$, $VacB_j$) combinations, where

the Ath element in the ordered list of combinations < ($VacA_i$, $VacB_j$), and

($VacA_i$, $VacB_j$) < the Bth element in the ordered list of combinations.

Input Limits and Constraints

- $1 \le n \le 10^5$
- $1 \le m \le 10^5$
- $1 \le VacA_i \le 10^9$ ($1 \le i \le n$)
- $1 \le VacB_i \le 10^9$ ($1 \le i \le m$)
- $1 \le A < B \le n \cdot m$
- $0 \le B - A \le 10^5$

Sample Input

```
3 5 2 5
1
5
6
3
3
3
4
6
```

Output for Sample Input

```
1
(1,4)
```

Explanation

The 2nd element in the ordered list of combinations is (1, 2), with a value of 1 + 3 = 4.

The 5th element in the ordered list of combinations is (2, 1), with an effectiveness of 5 + 3 = 8.

The only value between the two boundaries, which is strictly greater than 4 and strictly less than 8, is (1, 4) with an effectiveness value of 5.

Mr. G and Mr. D

Mr. G and Mr. D want to traverse a square matrix of size $n \times n$ in horizontal (left to right or vice versa) or vertical (top to bottom or vice versa) directions. The cost of a crossing is equal to the sum of the numbers on the chosen route. The horizontal crossing is made through the area indicated in figure A, and the vertical one according to figure B. The crossing rules are as follows: (i) the route starts at the border of the matrix (first/last row or first/last column), (ii) at each step, it forwards a row or a column, (iii) the consecutive cells on the route must be adjacent (they have a common side or corner). Mr. D is able to find the optimal path in the chosen direction (the sum of the numbers in the crossed cells is minimal). On the other hand, Mr. G follows a simplistic method: (i) he chooses the cell with the minimum value on the starting row / column, (ii) at each step, he chooses the neighboring cell with the minimum value. The advantage of Mr. G is that being faster he chooses the crossing direction firstly. For example, if Mr. G chooses horizontal crossing (from left to right or vice versa), then Mr. D must cross the matrix in the vertical direction (from top to bottom or vice versa).

Is there a guarantee that Mr. D can cross the matrix at a lower cost than Mr. G?

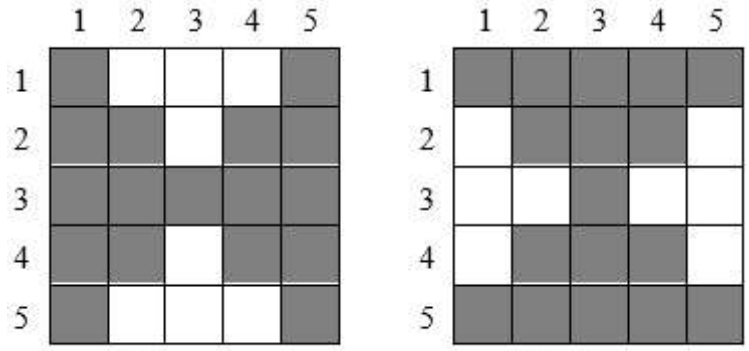


Figure A

Figure B

Input Specification

The first line of the input contains n, the size of the square matrix ($1 < n < 100$, n is odd). Each of the next n lines contains n positive integers (≤ 100), separated by one or more spaces, giving the elements of the matrix.

Output Specification

Print one line to the standard output, containing "YES" if Mr. D can cross the matrix at a lower cost than Mr. G, otherwise "NO".

Sample Input

```
5
1 2 4 5 6
8 9 3 5 6
2 8 6 6 3
7 3 5 9 2
9 4 6 3 1
```

Output for Sample Input

```
YES
```

Explanation

One example for the cost of crossing the matrix horizontally (from right to left) for Mr. G is 21 (Figure A. Mr. G horizontal), while for Mr. D, the cost of crossing the matrix vertically from bottom to top can be 18 (Figure B. Mr. D vertical).

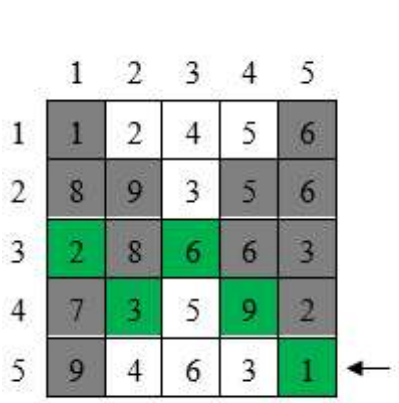


Figure A. Mr. G horizontal

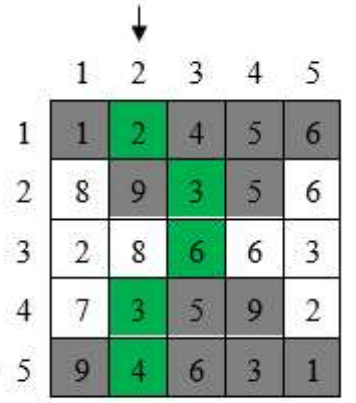


Figure B. Mr. D vertical

Vice versa, if Mr. G crosses the matrix vertically from bottom to top, the cost is 21 (Figure B. Mr. G vertical), while for Mr. D, crossing horizontally from left to right, the sum is 19 (Figure A. Mr. D horizontal).

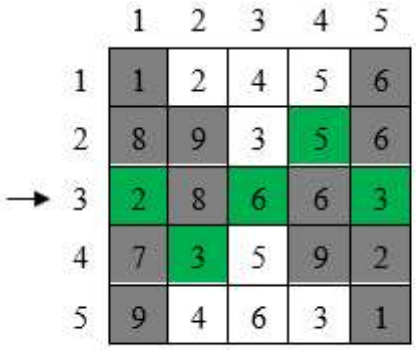


Figure A. Mr. D horizontal

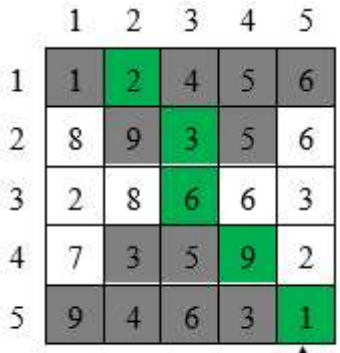


Figure B. Mr. G vertical

In both cases, Mr. D can cross the matrix at a lower cost than Mr. G, therefore the expected output is “YES”.

Lakes

You recently came across an ancient map. Thousands of lakes emerged on the map, both smaller and larger. You couldn't help but think about calculating the probability that, if you swim across an arbitrary lake from west to east starting from any given point, you swim across one of the largest lakes. Write a program to answer your torturous question!

Input Specification

The input contains several test cases. Each test case starts with a line containing three integers: the height of the map (N, $1 \le N \le 100$), the width of the map (M, $1 \le M \le 100$), and the number of crossed distances (D, $1 \le D \le 10$). Each of the next N lines consists of M characters: the "." (dot) character denotes land, the "O" (capital O) character denotes a piece of a lake. Pieces of the same lake are adjacent only by their four sides.

Each of the next D lines contains a single integer: the distances you take when crossing a lake.

The end of the input is denoted by a line containing three zeros.

Output Specification

For each distance read from the input, print one line to the standard output, containing a fraction in the form A/B, where A and B are relative primes. The value of the fraction should give the probability for the given distance. If you surely don't swim across one of the largest lakes, then print 0/1 to the output.

The outputs for each distance should be separated by a blank line.

Sample Input

```
8 10 4
..........
.O..OO.OO.
....OO.OO.
.OO.......
......OOO.
.OOO......
.....OOOO.
..........
1
2
3
4
8 10 4
..........
.O..OO.OO.
....OO.OO.
.OO.......
.......OO.
.OOO......
.O...OOOO.
..........
1
2
3
4
0 0 0
```

Output for Sample Input

```
0/1
2/3
0/1
1/1

1/2
1/2
1/1
1/1
```